DATE : / / SUB : نظریه زبان ها و ماشین ها

تعاریف های پایه :

الفبا : مجموعه ای محدود و غیرخالی از نماد ها (حروف) که با سیگما Σ نشان می دهیم

$\Sigma = \{a, b\}$, $\Sigma = \{0, 1\}$, $\Sigma = \{i, j, k\}$

رشته : هر دنباله ای به طول محدود از اعضای های الفبا .

$\Sigma = \{a, b\} \longrightarrow a, abab, aabb$ رشته هایی از Σ هستند .

$|x|$: اندازه یا تعداد نماد x

رشته خالی : ε یا λ $\quad |\varepsilon| = |\lambda| = 0$

$(ab)^3 = ababab$

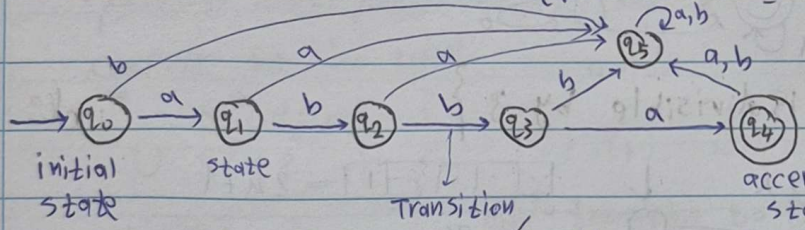
$a^n \leftarrow$ n بار حرف a را کنار هم نگه داریم

$\Sigma^* \leftarrow$ مجموعه ای از تمام رشته های ممکن از Σ (معمولاً تعدادش بینهایته)

اگر x و y دو تا رشته باشند xy یک رشته جدید است .

زبان : مجموعه ای رشته ها (بیش از تعداد ای الفبا)

مثال :



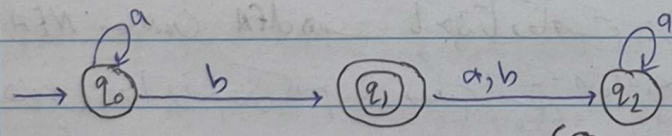
Alphabet $\Sigma = \{a, b\}$

برای اکسپت شدن یک رشته : ① همه ورودی ها باید اصلی بشوند ② آخرین استیت باید accepting باشد

برای ریجکت شدن یک رشته : ① همه ورودی ها باید اصلی بشوند ② آخرین استیت باید نو اکسپت استیت نباشد

Language of DFA M: $L(M)$ نشان می دهیم که شامل همه رشته هایی است که اکسپت شدن

مثال : $L = \{a^n b : n \geq 0\}$



Formal Definition : $M = (Q, \Sigma, \delta, q_0, F)$

transition function $= \delta$, $(\lambda \notin \Sigma)$ input alphabet $= \Sigma$, set of state $= Q$

$\delta(q, x) = q'$ $\quad F \subseteq Q \leftarrow$ set of accepting state $= F$, Initial state $= q_0$

$q \xrightarrow{x} q'$

(برای مثالی که گفتیم) Transition Table for δ

states \ symbols	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$\delta^* : Q(q, w) = q'$

$\rightarrow \delta^*(q_0, ab) = q_2$

$\delta^*(q_1, bba) = q_4$

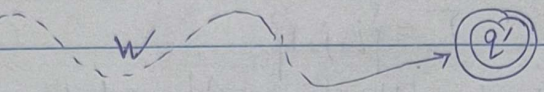
SOBHAN

30

DATE : / / SUB :

مهم ترین تعریف: for a DFA $M=(Q, \Sigma, \delta, q_0, F)$

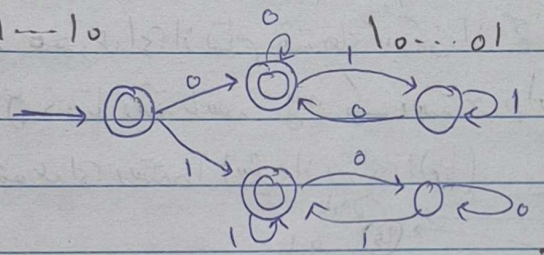
language accepted by M: $L(M)=\{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$



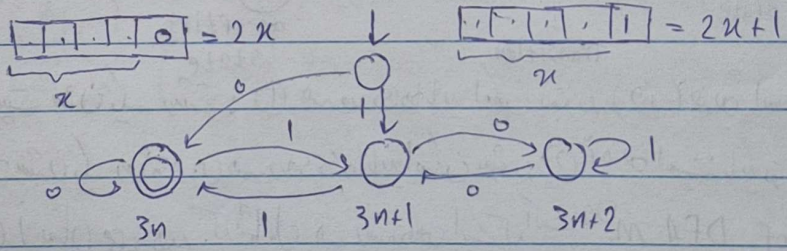
$L(M)=L$ زبان منظم است اگر dfa M برای آن داشته باشیم که (زبان منظم) : Regular languages

برای مثال زبان $L=\{a^n b^n : n>0\}$ غیر منظم است چون نمی توانیم برای آن dfa رسم کنیم

مثال خوب: $L_1=\{w | w \text{ contains equal number of substring } 01, 10\}$

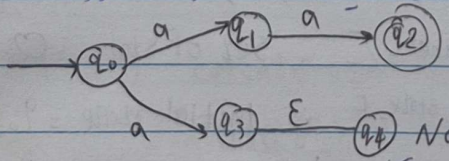


مثال: $L_2=\{w \text{ in } \{0,1\}^* | w \text{ is divisible by } 3\}$



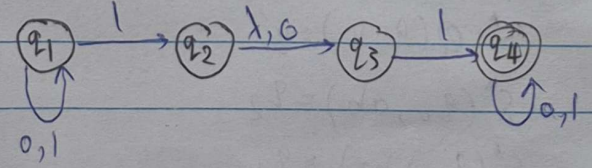
NFA ← همون dfa هست با چند تفاوت: ① لزوما برای هر است همه transition (No transition) ② ممکن است خروجی را رسم نکنیم ③ ممکن است ε یا λ داشته باشیم

از یک state دو تا transition هم نام به دو است مختلف داده شود.



اگر مسیری نتوانیم پیدا کنیم که به پذیرش ما به اکسپت است ختم شود میگیم که NFA درست است.

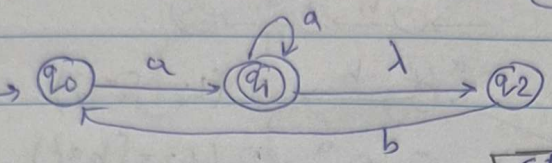
مثال ← w = 0101



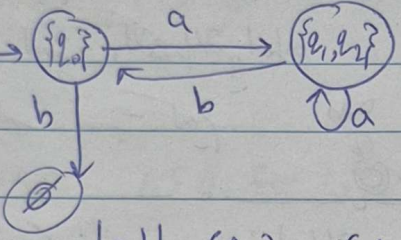
SOBHAN

DATE : / / SUB :

خیلی آسونه فقط قطعاً می‌ترسی conversion NFA to DFA

NFA → 

symbols → / state ↓	a	b
q_0	$\{q_1, q_2\}$	$\emptyset$
q_1	$\{q_1, q_2\}$	$\{q_0\}$
q_2	$\emptyset$	$\{q_0\}$

DFA M' → 

$q_1 \xrightarrow{a} (q_1, q_2)$ — $q_2 \xrightarrow{a} \emptyset$ → union → (q_1, q_2)

$q_1 \xrightarrow{b} q_0$ — $q_2 \xrightarrow{b} q_0$ → (q_0)

مراحل نوشتاری: ① همون استیت اولیه NFA (q_0) رو ابتدا می‌نویسیم.

② برای همه استیت‌های DFA $\{q_i, q_j, \dots, q_m\}$ مراحل این کار رو می‌کنیم:

compute in the NFA

$$\left.\begin{array}{l} \delta^*(q_i, a) \\ \cup\ \delta^*(q_j, a) \\ \dots \\ \cup\ \delta^*(q_m, a) \end{array}\right\} = \{q'_k, q'_l, \dots, q'_n\}$$

union

add transition to DFA

$$\delta(\{q_i, q_j, \dots q_m\}, a) = \{q'_k, q'_l, \dots, q'_n\}$$

③ مرحله 2 را برای همه استیت‌های DFA تکرار می‌کنیم و زمانی الگوریتم تمام می‌شود که دیگر dfa ای نوشیم.

④ for any DFA state $\{q_i, q_j, \dots, q_m\}$ if some q_j is accepting state in NFA Then, $\{q_i, q_j, \dots, q_m\}$ is accepting state in DFA

ویژگی‌های زبان منظم

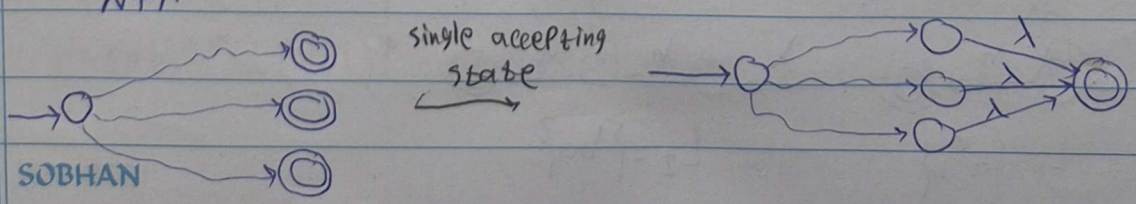
union: $L_1 \cup L_2$ concatenation: $L_1 L_2$ star: L_1^* Reversal: L_1^R

complement: $\overline{L_1}$ *Intersection: $L_1 \cap L_2$

می‌خواهیم ثابت کنیم که زبان منظم به این عملیات بسته هستند یعنی اگر L_1, L_2 منظم باشند هر کدوم از این عملیات‌ها هم زبان منظم هستند.

A useful transformation: نکته: سعی می‌کنیم که فقط یک accept state داشته باشیم

NFA

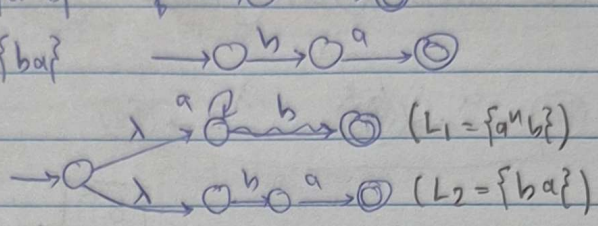


SOBHAN

DATE : / / SUB :

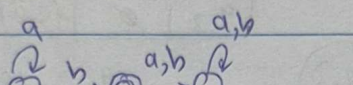
① union example: $L_1 = \{a^n b\}$ $L_2 = \{ba\}$

NFA for $L_1 \cup L_2$ →



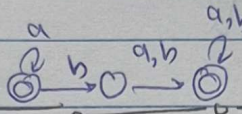
② Complement

$L_1 = \{a^n b\}$



* فقط برای DFA انجام میشه
باید هرجا اکسپت است دالتا باکپ را بدان
و ساده ها را اکسپت است کنیم.

$\overline{L_1} = \{a,b\}^* - \{a^n b\}$



③ Intersection

میخواهیم نشان بدیم که اگر L_1 و L_2 منظم باشند، $L_1 \cap L_2$ هم منظم است.

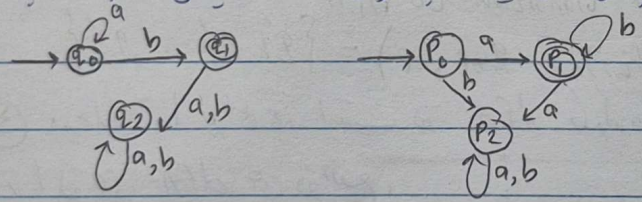
دمورگان → $L_1 \cap L_2 = \overline{L_1} \cup \overline{L_2}$

منظم = منظم → $\overline{L_1} \cup \overline{L_2}$ = منظم → $\overline{\overline{L_1} \cup \overline{L_2}}$ = منظم → $L_1 \cap L_2$ = منظم

L_1, L_2 = منظم → $\overline{L_1}, \overline{L_2}$ = منظم → $\overline{L_1} \cup \overline{L_2}$ = منظم → $\overline{\overline{L_1} \cup \overline{L_2}}$ = منظم → $L_1 \cap L_2$ = منظم

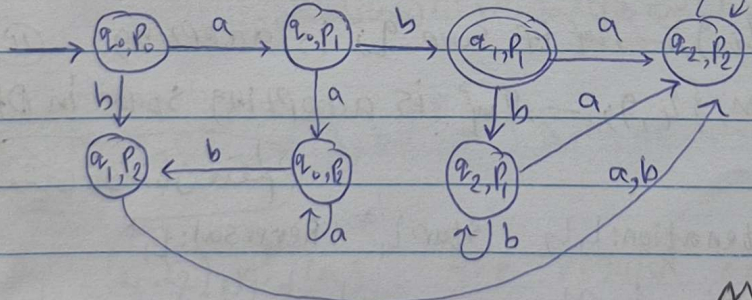
الگوریتم پیدا کردن اشتراک: ① است آغازیه را q_0, p_0 در نظر میگیریم ② مثل قبل NFA و DFA کامل میکنیم
③ در صورتی که همه المان های یک استیت توی DFA اولیه‌اش اکسپت است بود آنرا اکسپت میکنیم

مثال متوجه میشی

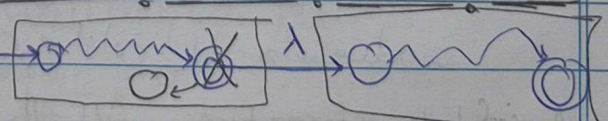


باهم قطعاً منفی:

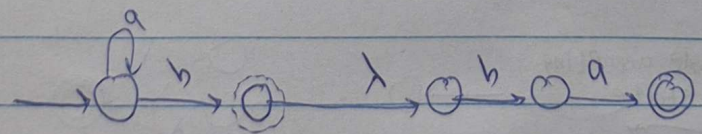
$L = \{a^n b\} \cap \{ab^n\} = \{ab\}$



④ Concatenation NFA for $L_1 L_2$



مثال → $L_1 L_2 = \{a^n b\}\{ba\} = \{a^n bba\}$

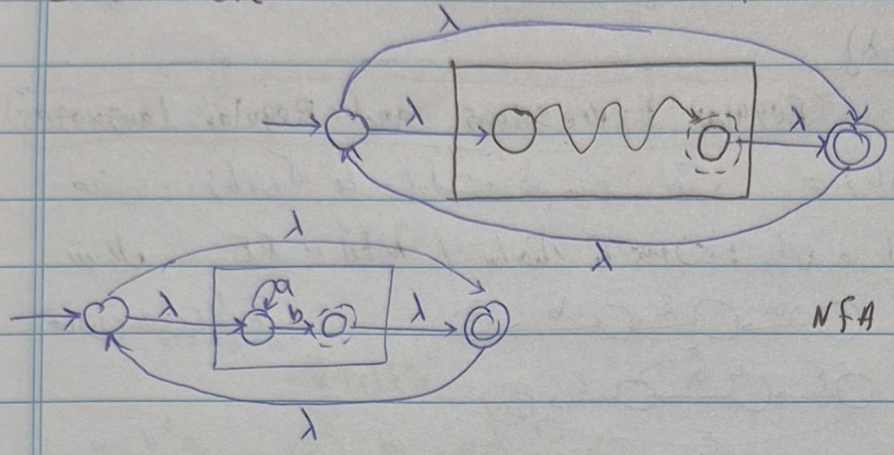


$L_1 = \{a^n b\}$ $L_2 = \{ba\}$

SOBHAN

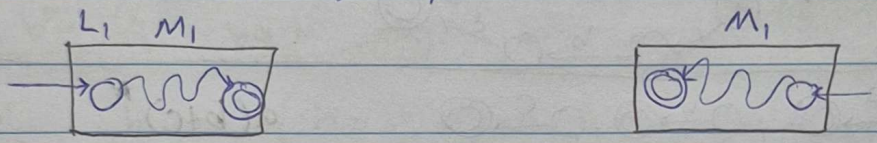
DATE : / / SUB :

(5) star operation NFA for L_1^*



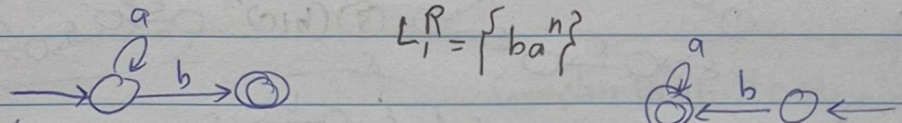
مثال: NFA for $L_1^* = \{a^n b\}^*$

(6) Reverse NFA for L_1^R



① همه‌ی فلش‌ها رو برعکس می‌کنیم ② استیت اولیه رو اکسپت می‌کنیم و برعکس

مثال: $L_1 = \{a^n b\}$ $L_1^R = \{ba^n\}$



چون ممکنه اولش حتا اکسپت استیت داشته باشیم اول همش رو با λ میریم تو یکی استیت بعدش برعکس میکنیم

Regula Expressions ③ NFA ② dfa ① نماسیتی های استاندارد زبان های منظم Regulal Expressions

مثال → $(a+b.c)^* = \{a, bc\}^* = \{\lambda, a, bc, aa, abc, bca, \dots\}$

$(r_1+r_2), (r_1.r_2), r_1^*, (r_1)$ Are regulal expressions

$L(r_1+r_2) = L(r_1) \cup L(r_2)$ / $L(r_1.r_2) = L(r_1)L(r_2)$ / $L(r_1^*) = (L(r_1))^*$

مثال (بدیهی هستا ولی باید اینجوری بنویسیم) $(a+b).a^*$

$L((a+b).a^*) = L((a+b))L(a^*) = (L(a) \cup L(b))(L(a))^* = (\{a\} \cup \{b\})(\{a\})^*$

$= \{a,b\}\{\lambda, a, aa, aaa, \dots\} = \{a, aa, aaa, \dots, b, ba, baa, \dots\}$

$r = (0+1)^* 00 (0+1)^*$ $L(r) = \{$all string containing substring $00\}$

60

DATE : / / SUB :

سومین مثال: $L=\{\text{all strings without substring } oo\}$

$r_1=(1+01)^*(0+\lambda)$

Regular Expressions and Regular Languages

تعریف: زبانی که با عبارات منظم ساخته شود = زبان منظم (انباشتن رو تو اصطلاح پایتون)

تبدیل RE به NFA با ساختار تامپسون: برای هر اپراتور از ع کمک می‌گیریم

قانون‌های پایه:

NFA for ab →

NFA for a^* →

NFA for a+b →

مثال: $a(b+c)^*$

① a,b,c

② b+c

③ $(b+c)^*$

④ $a(b+c)^*$

البته که آدم عاقل اینو میفوسیه:

حالا تبدیل NFA به RE:

مثال →

برای حل این مسائل باید GNFA سازیم، حالا این چیه؟ دوتا ویژگی داره

① استیت آغازین که هیچ incoming نپذیرفته باشه ② استیت آخری، یکی باشه، outgoing نداشته باشه

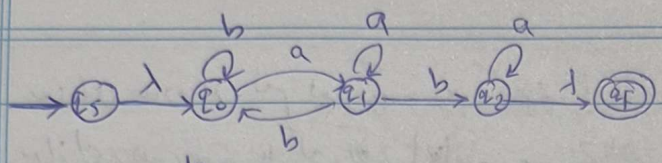
مثال، اگه داشته باشیم:

GNFA میشه:

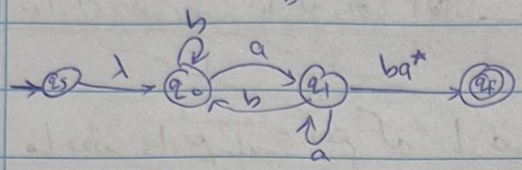
Ripping کردن:

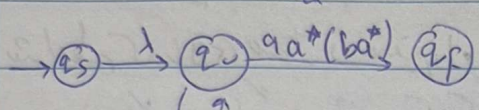
DATE : / / SUB :

مثال تبدیل NFA به عبارت منظم:



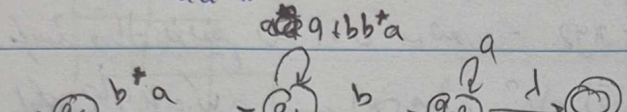
با حذف q_2 مسئله رو حل می‌کنیم:

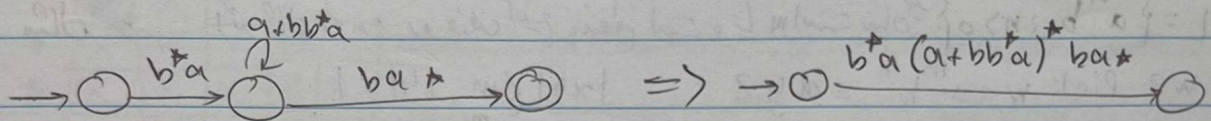




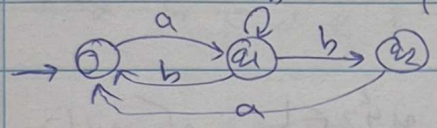
$\rightarrow (q_s) \xrightarrow{(b+aa^*b)^*aa^*(ba^*)} (q_f)$

اگر با حذف q_1 حل کنیم:





مثال: (اگر راه حل مخصوصی ویژه 7، دو دقیقه آخر رو ببین)



حواب نهایی $(ab^*(b+ba))^*$

اثبات اینکه بعضی از زبان‌ها منظم نیستند

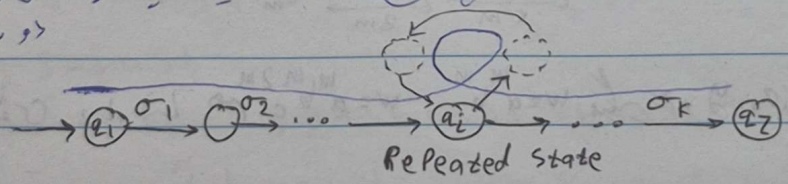
$\{a^n b^n : n>0\}$

$\{vv^R : v\in\{a,b\}^*\}$

مثال:

ایده اثبات: Pumping Lemma، لونه کبوتری

ارتباط لونه کبوتری با dfa ← اگر dfa، n استیت داشته باشد و رشته ورودی مان w باشد، اگر $|w|\geq n$ باشد آنگاه حداقل یک استیت را حداقل دو بار میبینیم



فرض کن رشته متن w است که $|w|\geq n$، q_i حداقل 2 بار تکرار شده ادعا می‌کنیم که میشه $w=xyz$ نوشت به طوری که x شامل استیت اول تا q_i بار، y بخش تکرار شده (از q_i تا q_i) بار، z از q_i تا آخر باره.

$x=\sigma_1\dots\sigma_i$

$y=\sigma_{i+1}\dots\sigma_j$

$z=\sigma_{j+1}\dots\sigma_k$

حالا بدیهی است که اگر بیشت y را i بار تکرار کنیم باز xy^iz اکسپت میشه $\infty>i\geq 0$

DATE : / / SUB :

برزبان ساده تر لم پمپینگ رو میگم: برمایه زبان منظم L ما m است میدن و ما میتونیم
برازای هر رشته w که $|w| \geq m$ ، w=xyz بنویسیم که $|xy| \leq m$ ، $|y| \geq 1$
$xy^i z \in L \quad i=0,1,2,\dots$

ما معمولا داریم ثابت کنیم که L غیرمنظم هست ، واسه این برهان خلف میزنیم و بعد
شرایطی که بالای صفحه نوشتیم رو اونش اعمال می کنیم و بهتناقض میرسیم .

★ یعنی اگه برمایین ثابت کن که L نامنظمه ما باید اول یک رشته w که $|w| \geq n$
باشه مثال بزنیم بعد اونو به صورت xyz بنویسیم بعد یک نمثال بزنیم و بگیم که
$L \not\ni xy^i z = w$ ← این میشه مثال نقض ✓

مثال : اگه مثال دوبرو رو فرض کنیم منظم است و با m است مثال میزنم $L=\{a^n b^n : n \geq 0\}$

we Pick $w = a^m b^m = xyz \qquad |xy| \leq m, |y| \geq 1$

چون $|xy| \leq m$ $\quad w = xyz = a^m b^m = \underbrace{a\cdots aa\cdots aa}_{x\ y}\underbrace{\cdots a}_{z}\underbrace{b\cdots b}_{m}$

$\longrightarrow x=a^{k_1}, y=a^{k_2}, z=a^{k_3}b^m$

$w' = xz = a^{k_1+k_3} b^m$ ← میدانیم که $xy^i z \in L$ حالا که $i=0$ میزنیم

ولی واضحه $k_1+k_3 < m$ چون $k_2 \geq 1$ پس $w' \notin L$ ناقض ✓

مثال: اثبات کنید زبان های زیر نامنظم است :

$L_1 = \{rr^R : r \in \Sigma^*\} \qquad L_2 = \{a^n b^{L} c^{n+L} : n, L \geq 0\} \qquad L_3 = \{w \in \{a,b\}^* \mid n_a(w) = n_b(w)\}$

$L_1 \rightarrow w = xyz \qquad w = a^m b^m b^m a^m \rightarrow \underbrace{aa\cdots a}_{x}\underbrace{bb\cdots b}_{y}\underbrace{a\cdots a}_{z}$

$|xy| \leq m \qquad |y| \geq 1$

$w' = xz \xrightarrow{i=0} \underbrace{aa\cdots a}_{<m}\underbrace{bb\cdots b}_{2m}\underbrace{a\cdots a}_{m}$ به فرم rr^R نیست

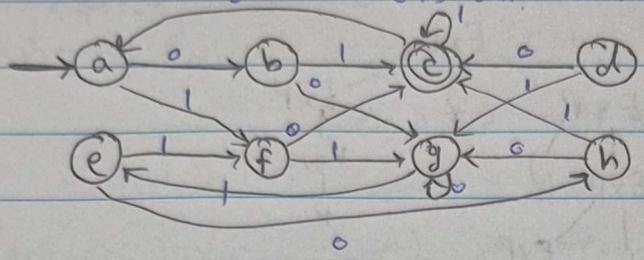
قرن 2 و 3 هم $w = a^m b^m$ ، $w = a^m b^{m+2m} c$ میگیریم و y رو حذف می کنیم .

SOBHAN

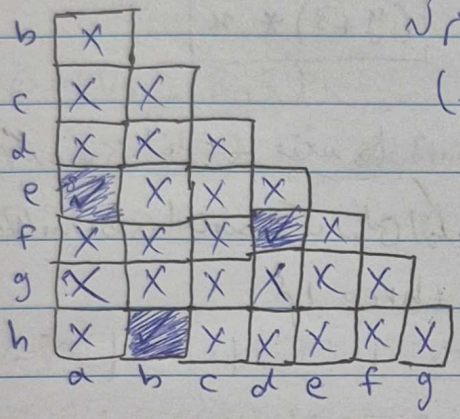
DATE : / / SUB :

DFA Minimization

قبل از اندازه آسون . قبل با مثال میفهمی ✓

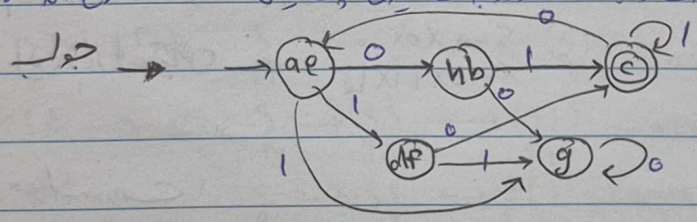


الان 8 تا استیت داریم باید جدول
بکشیم که همه $\binom{8}{2}$ حالت رو پوشش بده:



حالا الگوریتم اینه که اگر حالت a,b رو می خواهیم بررسی کنیم که
هم نوع باشن (یعنی یا هردو accept باشن یا هردو نباشن)
و با تو ترانزیشن یکی به استیت های همنوع بروند.
اگر همنوع نرفتن همون لحظه X می زنیم ولی اگر به همنوع
رفتن صبر می کنیم ببینیم که در آخر این 2 تا استیت
نهایی را می توان یکی در نظر گرفت یا نه.

مثلاً الان a و h با 0 هردو به g میرن و با 1 هردو به c میرن پس یکی هستند و همنوع اند ✓



جدول

دترمینیک هم هست ولی NFA نیست.

★ گرامرهای مستقل از متن (CFG)

گرامرها (نمایش زبان و عبارات نامنظم):

ایده اصلی: بازگشتی:

$L = \{a^n b^n \mid n \geq 0\}$ $\quad w = a^n b^n = a\boxed{a^{n-1} b^{n-1}}b$

$\begin{cases} S \to aSb \\ S \to \varepsilon \end{cases}$ $\quad S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$

Variables (متغیرها): اینها نمادهایی هستند که در فرآیند تولید رشته ها جایگزین می شوند
و گسترش پیدا می کنند مثل S توی مثال بالا

Terminals (ترمینال ها): اینها کاراکترهایی هستند که در زبان نهایی ظاهر می شوند و دیگر
جایگزین نمی شوند. جزو جن گرامر قطعا شامل نمادهای ترمینال است. مثل a, b توی مثال بالا

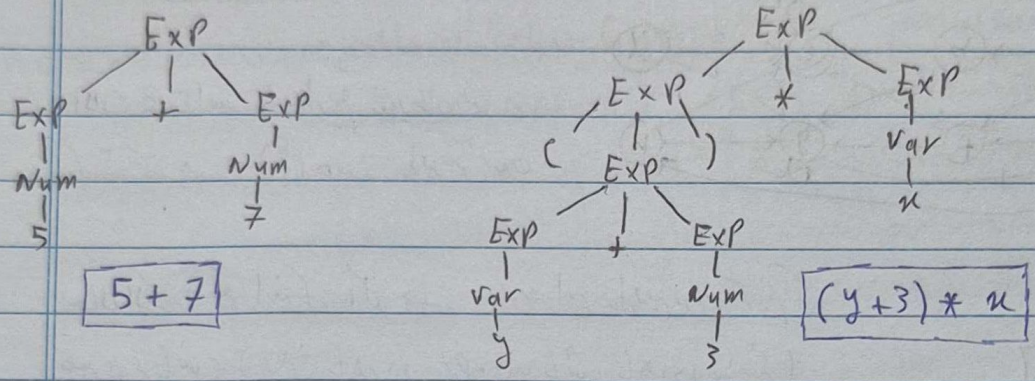
$Exp \to Var \mid Num \mid (Exp)$
$Exp \to Exp + Exp$
$Exp \to Exp * Exp$
$Var \to x \mid y \mid z$
$Num \to 0 \mid \ldots \mid 19$

- start symbol: Exp
variables: {Exp, Var, Num}
terminals: {x, y, z, 0, ..., 9, (,)}

SOBHAN

DATE : / / SUB :

همچنین این کلمه های متشکل از متن را میتوان به صورت درخت نشان داد که از چپ به راست آنرا میخوانیم.



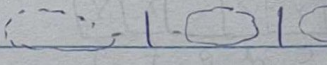
گره های داخلی = متغیرها برگ ها = ترمینال ها

حالا مثال های این مبحث که برای اینکه همشکل هارو ببینی و بدونی ما از چه موضوعی رو بین 42

① $L=\{w \in \{0,1\}^* \mid w \text{ contain at least three } 1\}$

$S \to X1X1X1X$

$X \to 0X \mid 1X \mid \varepsilon$

$(0+1)^* \to$ 

② $L=\{w \in \{0,1\}^* \mid \text{length of } w \text{ is odd and middle symbol is } 0\}$

$S \to X0X \to 0X0X \to 0\varepsilon0X \to 00\varepsilon \to 00$ (X حون)

این کار اشتباس ← $S \to X0X$, $X \to 0X \mid 1X \mid \varepsilon$

جواب درست:

$S \to 0S0 \mid 0S1 \mid 1S0 \mid 1S1 \mid 0$

③ $L=\{0^n 1^m 0^{2n} \mid n,m \geq 0\}$

جواب من: $S=1S \mid 0S00 \mid S$ / جواب اون:

$S \to 0S00 \mid X$

$X \to 1X \mid \varepsilon$

④ $L=\{w \in \{0,1\}^* \mid w \text{ starts and ends with same symbol}\}$

$S \to 0X0 \mid 1X1 \mid \varepsilon$, $X \to 0X \mid 1X \mid \varepsilon$

⑤ $L=\{0^i 1^j \# 0^k 1^\ell \mid i<j \text{ and } k>\ell\}$ $X \# Y$

$S \to X \# Y$, $E \to 0E1 \mid \varepsilon$, $X \to X1 \mid E1$, $Y \to 0Y \mid 0E$ جواب اون ←

$S \to X \# Y$, $X \to 1 \mid 0X1 \mid X1$, $Y \to 0 \mid 0Y1 \mid 0Y$ جواب من ←

⑥ $L=\{w \in \{a,b\}^* \mid w \text{ contains more a's than b's}\}$

$S \to XaX$

$X \to aX \mid aXbX \mid bXaX \mid \varepsilon$

SOBHAN

DATE : / / SUB :

انواع گرامر :

0) گرامرهای نامحدود (unrestricted) : در این گرامر ها هیچ محدودیتی روی قوانین بازنویسی وجود ندارد.

1) گرامرهای حساس به متن (CSG) : $\alpha A\beta \to \alpha\gamma\beta$ نماد تو در تو

2) ★ گرامرهای مستقل از متن (CFG) : $A \to \gamma$: A متغیر است و گاما رشته ای از نماد های ترمینال و غیر ترمینال است می تواند تهی (ع) هم باشد. (همین مجموعی که داریم کار میکنیم)

3) گرامرهای منظم (Regular Grammer) $A \to aB \mid a$

Context-Free Grammar ← تعریف ریاضی : $G = (V, \Sigma, R, S)$

V: (variables) R = finite non-empty set of rules

Σ: (terminals) $S \in V$ start variable

مثال ← $L(G) = \{a^n b^n \mid n > 0\}$

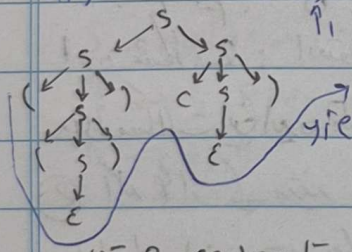
$V = \{S\}$, $\Sigma = \{a, b\}$, $R = \{S \to aSb \mid \varepsilon\}$ start symbol : S

Example. $G(\{S\}, \{(,)\}, R, S)$ $R = \{S \to SS \mid (S) \mid ()\}$ ① ② ③

Derivation: $S \to SS \to (S)S \to (S)() \to (())()$ (↑1 ↑2 ↑3 ↑3)

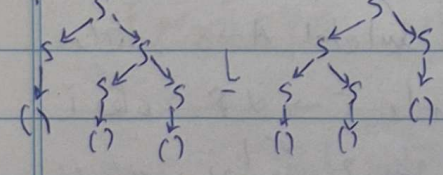
Left ~ : $S \to SS \to (S)S \to (())S \to (())()$ (↑1 ↑2 ↑3 ↑3)

Right ~ : $S \to SS \to S() \to (S)() \to (())()$ (↑1 ↑3 ↑2 ↑3)



← Parse tree

★ گرامر مبهم : گرامری مبهم است اگر برای yield آن بتوان دو تا Parse tree متفاوت رسم کرد یا دو تا Left Derivation یا دو تا Right Derivation متفاوت بتوان ارائه داد.



مثلاً برای ()()() ←

$S \to SS \to ()S \to ()SS \to ()()S \to ()()()$

$S \to SS \to SSS \to ()SS \to ()()S \to ()()()$

★ توجه نباید یک Right باشد و دیگری Left / هردو باید یک نوع باشند.

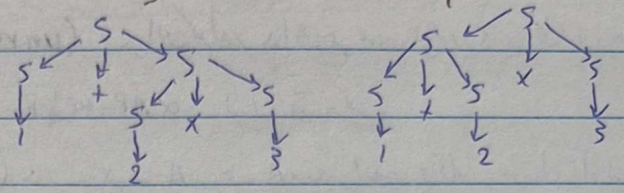
در ادامه ثابت میشه که هیچ الگوریتمی وجود ندارد که بتوان نشان داد یک گرامر مبهم هست یا نه. فقط حالتهای زیاد میشه مشخص

SOBHAN

اگر یک گرامر مبهم باشد اصلاً چیز خوبی نیست و کارها را هنگ می‌کند. مثلاً:

$S \to S+S \mid S*S \mid 0 \mid \cdots \mid 9$

$1+2\times 3$ yield $\to$

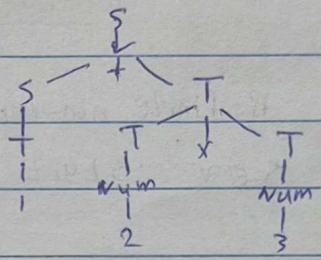


ما می‌توانیم از متغیرها و سطح‌های بتری استفاده کنیم تا به اولویت‌دهیم مثلاً تو مثال زیر ابتدا x‌ها انجام میشوند بعد +ها.

$S \to S+T \mid T$

$T \to T \times Num \mid Num$

$Num \to 0 \mid \cdots \mid 9$



در ادامه ثابت میشه که بعضی از گرامرها را این توان فقط از یک متغیرشان داد و باید از چند تا متغیر استفاده کنیم.

اگر بپرسند که آیا فلانی رشته جز گرامر هست یا نه: اگر عضو هست باید Derivation یا بالعکس سمتی نمایش بدیم و اگر عضو نیست این کار و نمیکنیم.

راه معقول اینه که بروت فورس بزنیم و همه حالات را بررسی کنیم ولی این راه دو تا مشکل عمده داره. ولی دو تا مشکل داره: ① بروت فورس ممکنه خیلی طولانی شه ② اگر ورودی توی زبان نباشه اونوقت از کجا بفهمیم؟ کی الگوریتم استوپ میشه؟

ایده مشکل 2: استوپ شه اگر: $|\text{input}| < |\text{derived string}|$

حالا مشکلات این ایده ① وجود ε موجب میشه سایز کوچیک بشود: $S \to OS1 \to OT1 \to 01$

② اگر ها از یک متغیر به یک متغیر تک تک بریم ممکنه تو لوپ بیوفتیم: $S \to T \to S \to T \to S \to \cdots$

حالا واسه این دو تا مشکل راه حل داریم:

راه مشکل اول: $A \to \varepsilon$ ها رو از بین میبریم (البته $\varepsilon \to S$) متغیر اول را از بین بدن بریم.

برای هر $A \to \varepsilon$ اول اینکه این قانون را حذف میکنیم و اگه داشته باشیم $C \to \alpha AB$ آنگاه $C \to \alpha B$ را به قانون‌ها اضافه می‌کنیم. و اگر چندتا A باشه همه حالات رو باید در نظر بگیریم $\leftarrow$ $C \to \alpha ABA\gamma : C \to \alpha BA\gamma \mid \alpha AB\gamma \mid \alpha B\gamma$

* صفحه بعدی یک مثال خفن از این الگوریتم می‌زنم و بعدش توضیحات رو میدم:

DATE: SUB:

★ توجه کن که به جز $S \to \varepsilon$ میشه $A \to B$ هم مثلاً حذف کرد
بقیه جزوه 12 از صفحه 11 هست آسونه ولی ببین

$S \to S$

$S \to ABCD \mid \varepsilon$
$A \to SaS$
$B \to \varepsilon$
$C \to CSE \mid \varepsilon$
$D \to CB \mid b$
$E \to aB$

remove $S \to \varepsilon$

مثال: مرحله اول اینه که یک $S \to S'$ به اول دستور اضافه کنیم تا بتوانیم $\varepsilon \to S$ را حذف کنیم.
$S' \to \varepsilon$ مشکلی ندارد.
بعد از این کار $\varepsilon \to ?$ ها رو حذف می کنیم در این مثال یعنی باید $S \to \varepsilon$ ، $B \to \varepsilon$ و $C \to \varepsilon$ ها رو حذف کنیم.
و اگر ؟ سمت راست باشد آن را باید حذف کنیم ← دقت کن:

Add: $S' \to \varepsilon$
$A \to Sa$
$A \to aS$
$A \to a$
$C \to CE$

remove $B \to \varepsilon$
Add: $S \to ACD$
$D \to C$
$E \to a$

remove $C \to \varepsilon$
Add: $S \to ABD$
$C \to SE$
$D \to B$
$C \to E$
$S \to AD$
$D \to \varepsilon$

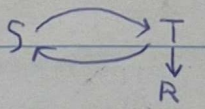
remove $D \to \varepsilon$
Add: $S \to ABC$
$S \to AC$
$S \to AB$
$S \to A$

نکته: وقتی داریم قوانین که به اپسیلون میره رو حذف می کنیم و قوانین جدید در نظر میگیریم در نهایت قوانین نهایی ما شامل این قوانین ابتدایی هم میشن. و همچنین در حذف اپسیلون باید به قوانین جدید هم توجه کنیم!

الگوریتم رفع مشکل لوپ:
گراف متغیرها را رسم می کنیم و به جای یک دور از یک حرف استفاده می کنیم:

Grammar:
$S \to 0S1 \mid 1S0S \mid T$
$T \to S \mid R \mid \varepsilon$
$R \to 0SR$

unit Production Graph



$S \to 0S1 \mid 1S0S$
$S \to R \mid \varepsilon$
$R \to 0SR$

S
↓
R

سپس به جای آها داریم:
$S = 0S1 \mid 1S0S \mid R \mid \varepsilon$ ← replace any ch
$R \to 0S1$

جهت مشکل اول حل شد ✓ یک الگوریتم وجود دارد به اسم CYK الگوریتم که نشون میده یک رشته در گرامر هست یا نه.
نکته این الگوریتم اینه که حتما باید قانون ها به این سه قانون باشه:

$A \to BC$
$A \to a$
$S \to \varepsilon$

واضحه که میشه همرو این شکلی کرد ← مثلاً اگه $A \to BCDe$ باشه
↳ $Ac \to c$, $Ae \to e$, $A = BA_cDA_e$ ⊗ / $A = BF$
$F = A_c G$
$G = DA_e$

SOBHAN
به این میگن فرم نرمال چامسکی

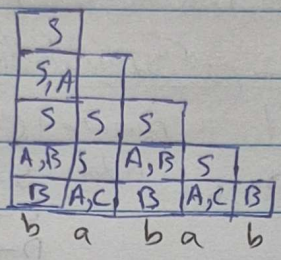
14

DATE : / / SUB :

حال الگوریتم چیه؟ توضیح ریاضیش و معنوی‌ش خیلی سخته.
با مثال توضیح میدم.

گرامر: $S \to AB \mid CB$
$A \to BA \mid a$
$B \to BC \mid b$
$C \to AC \mid a$

میخواهیم ببینیم که آیا babab جزء گرامر هست یا نه.



* باید مثالی حل کنیم (جدی خوب نیست توضیح بدم اگه یادت رفت ویدئو 12 رو ببین)

A:

توضیح سطر ①: باید ببینیم هرکدام از b, a ها سمت راست کدوم متغیرها هستند.

توضیح سطرهای دیگه: اینکاره عمل می‌کنیم که از ستون اول بالا میریم و

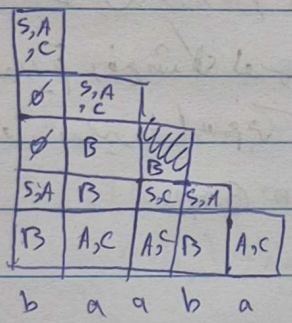


مثلا‌ی پایین می‌آییم و اینارو چک می‌کنیم که ببینیم ترکیب اینا تو سمت راست کدوم متغیر هست اگه خانه بالایی S بود یعنی زبان اون ورودی رایوشن میده ✓

مثال دیگر:

گرامر → $S \to AB \mid BC$
$A \to BA \mid a$
$B \to CC \mid b$
$C \to Ab \mid a$

$w = baaba$



تبدیل گرامر چپ به راست

توی اکثر جاها مثل کامپایلر Left-recursive خوب نیست و باید به راست تبدیل کنیم.
دو حالت داریم: ① direct یعنی مستقیم بینیم ② indirect بعد از چند مرحله تبدیل لوپ داره.

قاعده کلی direct: $A \to A\beta_1 \mid A\beta_2 \mid \dots \mid A\beta_n$, $A \to \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_k$

$\Rightarrow A \to \gamma_1 A' \mid \gamma_2 A' \mid \dots \mid \gamma_k A' \mid \gamma_1 \mid \dots \mid \gamma_k$, $A' \to \alpha\beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A' \mid \beta_1 \mid \dots \mid \beta_n$

یادآوری: به جای $X \to X\alpha$ میذاریم $X \to \beta$ و به جای $X' \to \alpha X'$ میذاریم $X \to \beta X'$

مثال:

$E \to E+T \mid T$, $E' \to +TE' \mid +T$, $T' \to *FT' \mid *F$
$T \to T*F \mid F$, $E \to TE' \mid T$, $T \to FT' \mid F$
$F \to (E) \mid a^n$

SOBHAN

DATE : / / SUB :

Indirect left Recursion

$A \to Ba \mid \varepsilon$
$B \to cb$
$C \to Ac$

$A \to Ba \to cba \to Acba \to \dots$

بعد از حذف مرحله متوجه لوپ میشویم.

$A \to Acba \mid \varepsilon$
$A' \to cbaA' \mid cba$ ✓ Right

ایده اولیه اینکه ابتدا به direct تبدیل کنیم بعد به
حالا برای این کار یک الگوریتم داریم که به همه متغیرها یک اندیس نامیریم تا همه به
فرم $A_1, A_2, \dots A_n$ باشن بعدش این دستور های تولی را حذف می کنیم یعنی مثل
$A_5 \to A_2 b$ ، بنابراین فقط دستورامون بصورتی هست ذو لوپ نداریم.

★ منظور از حذف کردن هم یعنی جایگذاری کردن

مثال:

$A_1 \to A_2 a \mid b$
$A_2 \to A_2 c \mid A_1 d \mid \varepsilon$

ابتدا $A_2 \to A_1 d$ را باید با جایگذاری ازین بریم

بین داریم $A_2 \to A_2 ad \mid bd$ حالا که direct شده مثل آدم حلش میکنیم ↙

$A_1 \to A_2 a \mid b$
$A_2 \to A_2 c \mid A_2 ad \mid bd \mid \varepsilon$

$\longrightarrow$

$A_1 \to A_2 a \mid b$
$A'_2 \to cA'_2 \mid adA'_2 \mid c \mid ad$
$A_2 \to bdA'_2 \mid bd \mid \varepsilon$

Greibach Normal Form

که $A \to x V^*$ بدون ساده یعنی هر دستور به یک ترمینال در سمت چپ و تعدادی متغیر باشد.

برای مثال این فرم ها قوی کریباخ نیستند: $A \to \varepsilon$ ، $A \to B$ ، $X \to A\alpha B$ ، $X \to AB$

راه حل تبدیل کردن به فرم کریباخ: ① اول همه Left recursion ها را حذف می کنیم.

$X \to Y\alpha$
$Y \to \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_n$ $\Rightarrow X \to \gamma_1 \alpha \mid \dots \mid \gamma_n \alpha$ ← ② اگر متغیری سمت چپ بود اونو جایگذاری می کنیم

مثال (من خودم):

$A_1 \to A_2 a$
$A_2 \to A_1 c \mid bA_1 \mid d$

① $A_1 \to A_2 a$
$A_2 \to A_2 ac \mid bA_1 \mid d$

$A_1 \to A_2 a$
$A_2 \to bA_1 A'_2 \mid dA'_2 \mid d \mid bA_1$
$A'_2 \to acA'_2 \mid ac$

② $A_1 \to bA_1 A'_2 a \mid dA'_2 a \mid da \mid bA_1 a$
$A_2 \to bA_1 A'_2 \mid dA'_2 \mid d \mid bA_1$
$A'_2 \to acA'_2 \mid ac$

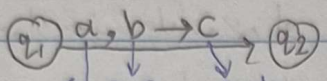
③ $A_1 \to bA_1 A'_2 Xa \mid dA'_2 Xa \mid dXa \mid bA_1 Xa$
$A_2 \to bA_1 A'_2 \mid dA'_2 \mid d \mid bA_1$
$A'_2 \to aXcA'_2 \mid aXc$

این واسه خروس
خودم بهتر حل میکنم
خودت خفن تری!

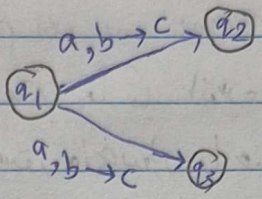
SOBHAN $Xa \to a$ ، $Xc \to c$

DATE : / / SUB :

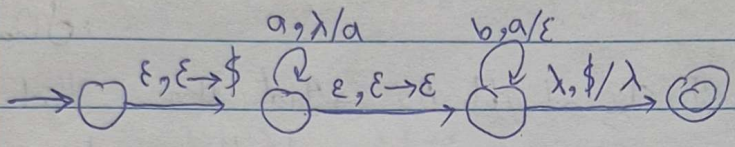
PDA حول Nfa و dfa كيف تحول Stack عبارة عن ذاكرة إضافية مع برامج ذهنية كبيرة.



Non-Deterministm:

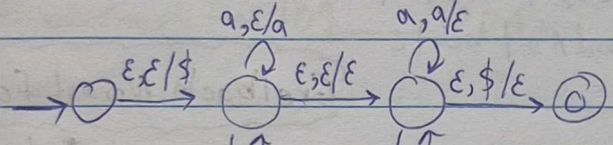


مثال: PDA M: $L(M)=\{a^n b^n : n \geq 0\}$



مثال آخر:

PDA M: $L(M)=\{vv^R : v \in \{a,b\}^*\}$

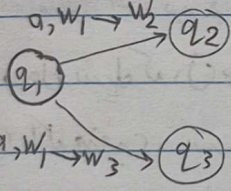


Formal Definition:

initial state, stack start symbol

$M=(Q, \Sigma, \Gamma, \delta, q_0, Z, F)$

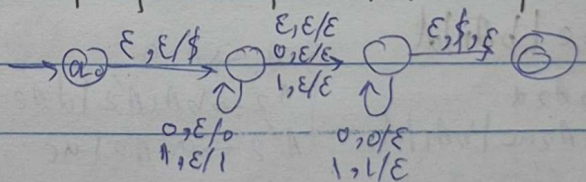
state — Input alphabet — stack alphabet — transition function — Accept states



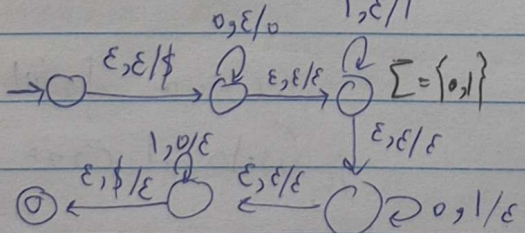
Transition function: $\delta(q_1, a, w_1) = \{(q_2, w_2), (q_3, w_3)\}$

مثال: $\Sigma=\{0,1\}$ $L=\{w \in \Sigma^* \mid w=w^R\}$

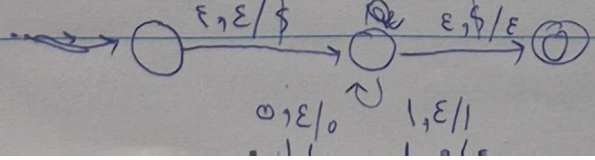
00100 01010 | 0110 | 01100



مثال: $\Sigma=\{0,1\}$ $L=\{0^n 1^m 0^m 1^n \mid n \geq 0, m \geq 0\}$



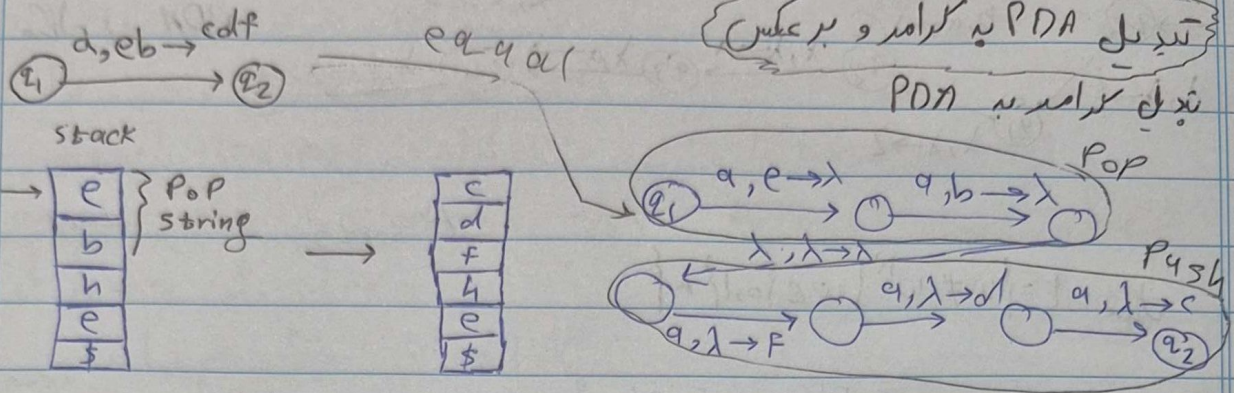
مثال: $\Sigma=\{0,1\}$ L = same number of 0's and 1's



SOBHAN

تبدیل PDA به گرامر و برعکس

تبدیل گرامر به PDA



اصلا" اصلا" سخت نیست مثال بین میزنم: برای تبدیل به PDA ما

Grammar

$S \to aSTb$

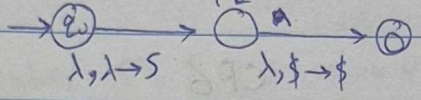
$S \to b$

$T \to Ta$

$T \to \lambda$

$\lambda,S \to aSTb$ $\lambda,T \to Ta$ $a,a \to \lambda$

$\lambda,S \to b$ $\lambda,T \to \lambda$ $b,b \to \lambda$



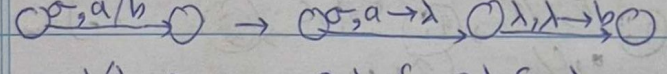
توضیح خلاصه: همون سه تا حالتی گرامر رو میذاریم و ترمینال ها رو هم Pop میکنیم.

تبدیل PDA به گرامر روش: PDA ← simplified PDA ← گرامر

اولین کاری که باید توی PDA کنیم: ① PDA فقط یک اکسپت استیت داره

② هر ترنزیشن یا Push هست یا Pop نباید هر دو باشن.

واسه مورد اول که از همه اکسپت استیت ها با $\varepsilon/\varepsilon/\varepsilon$ فلش میزنیم به یک استیت دیگه ✓



واسه مورد دوم:

ساختار گرامرها: variables: A_{q_i,q_j} یعنی از استیت q_i میره تا q_j

انواع استیت ها: ⓟ ⓠ ⓡ برای هر سه استیت:

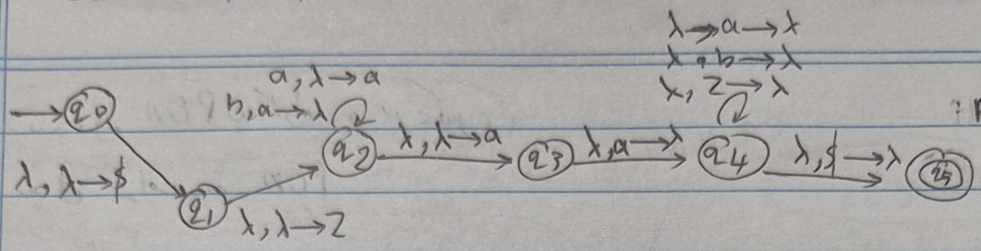
ⓠ: $A_{qq} \to \lambda$

$A_{pq} \to A_{pr} A_{rq}$

برای چینش زوج هایی که $p \xrightarrow{a,\lambda \to t} r \xrightarrow{b,t \to \lambda} q$ گرامر: $A_{pq} \to a A_{rs} b$

اگر q_F اکسپت استیت و q_0 استیت اولیه باشه گرامر $A_{q_0 q_F}$ میشه Start variable

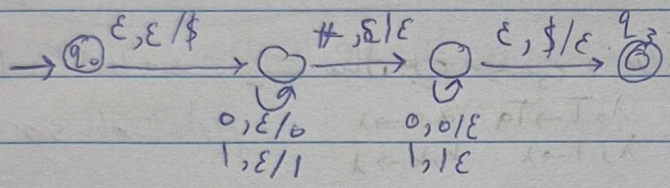
DATE : / / SUB :



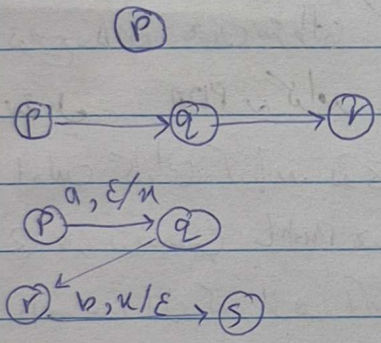
: PDA : مثال

: Start $A_{q_0 q_5}$

مثال : $L=\{w \# w^R \mid w \in \{0,1\}^*\}$



PDA



variables : A_{00}, A_{01}, A_{02}
A_{12}, A_{11}, A_{22}

start var : A_{02}

CFG

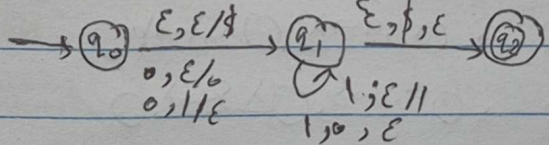
$A_{pp} \rightarrow \varepsilon$

$A_{pr} \rightarrow A_{pq} A_{qr}$

$A_{ps} \rightarrow a A_{qr} b$

$a = \varepsilon$ or $b = \varepsilon$

مثال :



$A_{02} \rightarrow A_{01} A_{12}$

$A_{01} \rightarrow A_{01} A_{11}$

$A_{12} \rightarrow A_{11} A_{12}$

$A_{11} \rightarrow A_{11} A_{11}$

$A_{11} \rightarrow 0 A_{11} 1$

$A_{11} \rightarrow 1 A_{11} 0$

$A_{02} \rightarrow A_{11}$

$A_{00} \rightarrow \varepsilon$ $A_{22} \rightarrow \varepsilon$

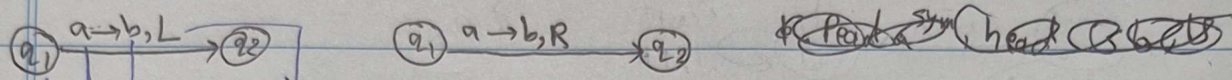
$A_{11} \rightarrow \varepsilon$

CamScanner

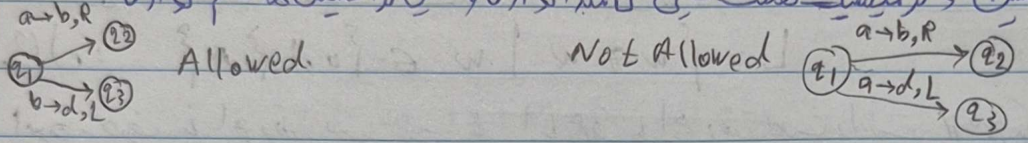
DATE : / / SUB :

جلسه ۱۲

Turing Machines یه چیزی هست که باهاش همه الگوریتم ها رو میشه نشون داد. که اینم مثل بقیه چیزاییه که باید یه یک واحد کنترول (استیت) داره و یک نوار بی نهایت. که این نوار یک head داره اول تو خونه اول هست و هر مرحله یا میره چپ یا راست.



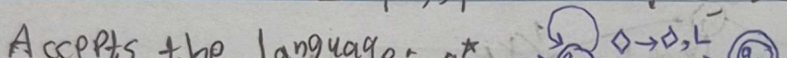
تورینگ ماشین دترمینستیک هست یعنی لامبدا نداره و این ترانزیشن ها هم نداره:



و اینکه اکسپت استیت نباید ترانزیشن داشته باشد.

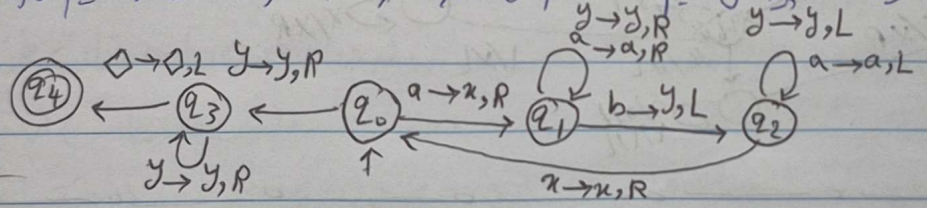
ساده ترین مثال ممکن تورینگ ماشین: $a \to a, R$

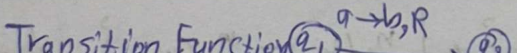
Input alphabet $\Sigma = \{a, b\}$

Accepts the language: a^* 

مثال روشن: $\{a^n b^n\} : n \geq 1$

ایده پاید: مقصد اینم هر a رو با هر b مچ کنیم: این کار رو تک تک میکنیم: چپ ترین a را میکنیم x و چپ ترین b رو پیدا می کنیم و میکنیم y. در آخر که a, b اصلا نباشه استیم تو ک



Transition Function 

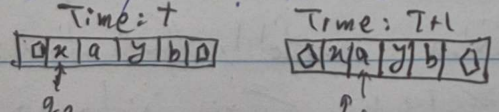
$\delta(q_1, a) = (q_2, b, R)$ ← Formal Definitions for Turing

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ → Accept

States ↓ Input alpha ↓ Tape alphabet ↓ transition function → initial

Configuration: فرض کن نوار اینه: abc و هد روی b ایستاده و ماشین در حالت q_2 قرار داره

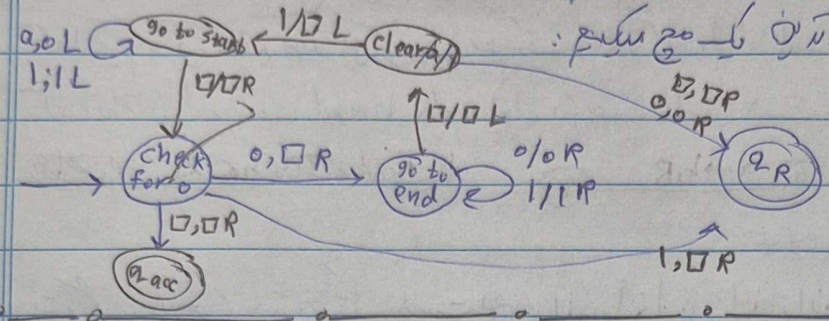
یعنی a رو و قبلا خونده/در حالت q_2 هست/b رو هم config به شکل زیر نوشته میشه: $a\, q_2\, bc$



A move: $q_2 xayb \vdash x q_0 ayb$

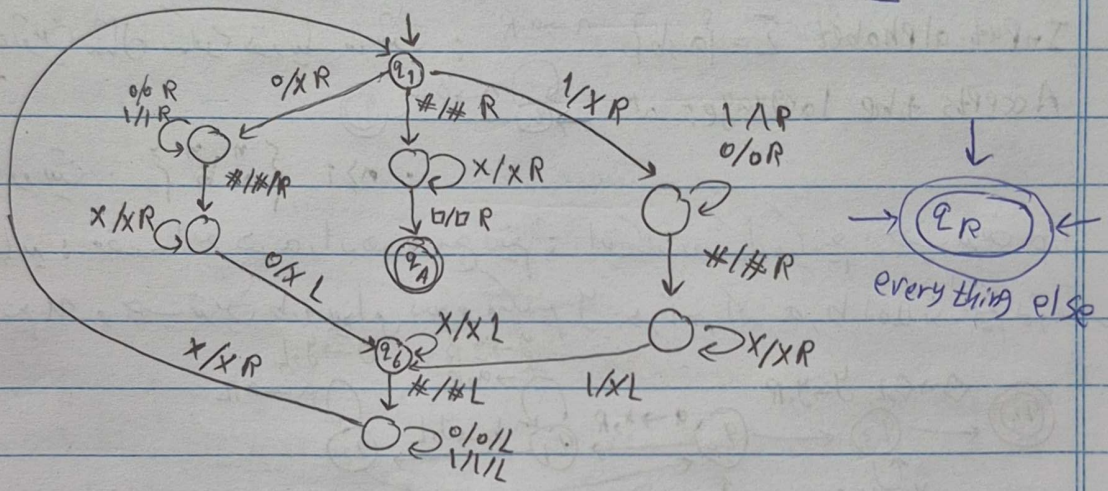
مثال های جزوه حفت: $L=\{0^n1^n \mid n\in IN\}$ $w=0w1$ بازگشتی حل میکنیم

چپ ترین صفر را با راست ترین یک مچ میکنیم:



مثال: $L=\{w\#w \mid w\in\{0,1\}^*\}$ ایده: حرف نام را رفته سمت چپ # را جرو نام رفته سمت راست # مچ میکنیم و X می گذاریم

$X\underline{1}00\#X1100 \rightarrow X\underline{X}00\#X1100 \rightarrow XX00\#X\underline{X}00$



مثال: $L=\{w\in\{a,b\}^* \mid n_a(w)=n_b(w)\}$ توجزوه بین آنبوته ✓ ایده: مچ ✓

مثال: $L=\{w\in a^* \mid |w|=3^n, n\geq 0\}$ توجزوه بین سخته ✗

ایده: هر مرحله w رو تقسیم بر ۳ میکنیم اگر اخیراً باقی مانده داشتیم R میشود.

مثال: $L=\{a^i b^j c^k \mid ij=k \text{ and } i,j,k>0\}$ ← خیلی سخت و مشکل ✓

ایده: a یا یکی را انتخاب بده: a را خط بزن بعد شمه b ها را خط بزن و به همون اندازه c ها هم خط بزن، بعد برگرد b ها را پیدا کردن و دوباره a دیگه بزن اینجوری axb بار c خط میزنیم

مثال: $L=\{\#x_1\#x_2\cdots\#x_m \mid x_i \in \{0,1\}^* \text{ and } x_i \neq x_j \text{ for every } i \neq j\}$

ایده: جالب‌ترین شاید این رو قدمت میکنیم دوستش رو آنه میکنیم که یعنی میتوانیم x_1 و x_2 را چک کنیم بعد آن رو تا نه میبریم بعدش قدمت رو یکی میاریم جلوتر و همین کار رو تکرار میکنیم تا همه دوتایی رو بررسی کنیم.

مثال: $L=\{a^p \mid p \text{ is a Prime number}\}$

ایده: عنایل اراتوستن: اولین عدد اول رو پیدا میکنیم (2) بعد همه مضارب دو را خط میزنیم بعدی میگیریم مضارب سه و... را خط میزنیم

راه حل: اول را # میزاریم و a آخرا را $ میزاریم

فرض کنید head روی موقعیت m باشد پس همه مضارب m را تبدیل به □ میکنیم و اگر $ مضرب m باشد او تبدیل به $* میکنیم

اول a که در موقعیت m هست رو قرار میدیم â و... خیلی قرو قمیدتر شو تا آخرین تو

(انواع مختلف ماشین تورینگ) variations of the Turing machine

در اصل آلن تورینگ یه نوع ساده از ماشین طراحی کرد. ولی بعدا محققان شیوه‌های مختلفی از اون طراحی کردن که ① از نظر قدرت محاسباتی معادل همون ماشین تورینگ اصلی هستن ② ولی ممکنه از نظر سرعت یا راحتی در طراحی الگوریتم فرق داشته باشن

① multi-tape Turing ma (ماشین تورینگ چند نواره): به جای یک نوار، چند نوار داره و هر نوار هد خودش رو داره

② two-way Infinite Tape (نوار دو طرفه بی نهایت):

③ Tm with a stay optio (با گزینه ماندن) در مدل اولیه، هد همیشه به چپ یا راست میرفت

④ Non-deterministic Tm در حالت عادی فقط یک حرکت داریم

---

① ماشین تورینگ با گزینه ماندن: خیلی واضحه که صرفا هروقت این ماشین از گزینه ماندن استفاده کردن:

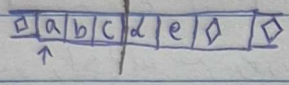
$q_1 \xrightarrow{a,b\ S} q_2$ یا همانطور که → $q_1 \xrightarrow{a/b\ L} q_2 \xrightarrow{x/x\ R} q_2$ for every symbol x

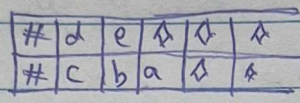
(II) شبیه سازی two way با ماشین تورینگ استاندارد:

بخش اول ① ← اینکه two way هر کاری که ماشین اولیه انجام میدهد رو انجام میده:

خیلی واضحه: مثلاً علامت # رو قبل چپ ترین ورودی رشته قرار میدهیم و تو هر استپ # →#,R → ○ → رو قرار میدهیم که بفهمیم از این چپ ترین تونیم برویم

بخش دوم ② ← یک جا این دو اول ورودی two way رو نشون میکنیم و بعدش ماشین استاندارد دو نواره تبدیل میکنیم



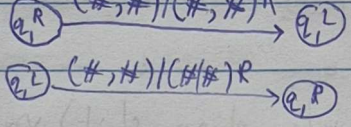


two-way → $(q_1)\ a \to g, R\ (q_2)$

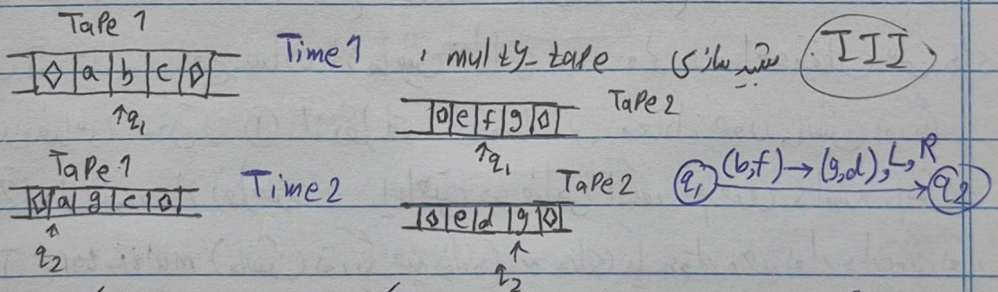
standard → Right Part: $(q_1^R)\ (a,x) \to (g,x), R\ (q_2^R)$

Left Part: $(q_1^L)\ (x,a) \to (x,g), L\ (q_2^L)$ for all tape symbols x

at the border:



(III) شبیه سازی multi-tape، اصل چیه؟



اثبات: اینکه این مدل هر کاری که مدل استاندارد میکنه که واضحه (فقط از یکی نوارش استفاده کنه)

حالا اثبات اینکه مدل اولیه میتونه کارایی این مدل رو کنه:

ما از یک نوار با چند track استفاده می‌کنیم طوری که Track i محتوای نوار i ام رو داشته باشه.

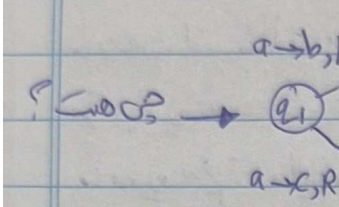
مراحل شبیه سازی: مرحله ① : برگردیم به نقطه مرجع ② هد هر نوار رو پیدا کن به صورت چپ

③ اعمال تراکنش: حالا که همه‌ی مثلاً: در حالت q هستیم، نوار 1 در a و نوار 2 در b

بر اساس جدول انتقال اصلی ماشین چند نواره: نماد ها رو تغییر بده / هدها رو حرکت بده (L/R)

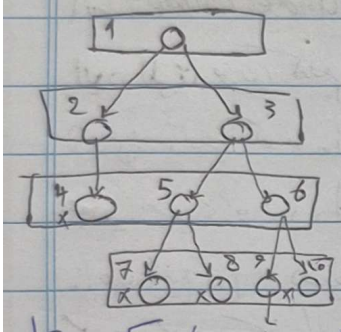
و به حالت جدید برو

(۱۷) شبیه سازی Non-deta- Tm



یک ظرف حقیه که واضحه: هر ماشین معمولی، نان دترمینیستیک هم هست یعنی نان دترمینیستیک میتونه همه کارای ماشین اولیه رو انجام بده ✓

حالا اینکه ماشین اولیه نان دترمینیستیک هم انجام میده:



درخت حالات رو میکشیم و اگه یک راه درست باشه (از ریشه تا برگ) یعنی زبان را اکسپت میکنه پس bfs میزنیم

مراحل: ① شروع از پیکربندی اولیه (ورودی و حالت شروع)

② تا وقتی یکی از مسیرها به جواب نرسیده: همه پیکربندی های مرحله در Level مرحله رو بررسی کن، اگر پیکربندی جدید داره: همه فرزندهارو بقیه سازی کن و تو لیست مسیر جدید کن، اگر یکی از پیکربندی ها به حالت قبول رسید ← قبول کن، اگر همه مسیرها به بن بست خوردن ← رد کن کدوم همون bfs هست.

ماشین تورینگ ها به ازای هر رشته ۳ خروجی روبرو رو دارند: {Loop, rej, Accept}

Decidable Language: زبان L تصمیم پذیر است اگر ماشین تورینگی وجود داشته باشد که برای هر رشته ورودی w، متوقف میشود. ① اگر $w \in L$ خروجی قبول و اگر $w \notin L$ خروجی رد میشود.

Recognizable Language: زبان L تشخیص پذیر است اگر ماشین تورینگی وجود داشته باشد که: اگر $w \in L$ حتما متوقف می شود و قبول میکند و اگر $w \notin L$، ممکن است رد کند یا تو لوپ بیوفته

یکیش مثال هست که نشون میده زبان diagonable هست یا نه که تو جزوه حجت بین

Reduction کاهش یه تکنیک که میگه: «برای حل یه مسأله جدید، اون رو به مسأله که از قبل میدونیم چطوری حل میشه، تبدیل کنیم» به طور خاص از ریداکشن برای نشون دادن اینکه یک زبان undecidable هست، استفاده می کنیم.

ما بعضی از مسائل معروف رو حواش رو میدونیم ← مثلاً میدونیم A_{TM} (ماشین تورینگی که
شون میده یک رشته عضو اون ماشین تورینگ هست یا نه)
ما میدونیم که این ها مساله یک مساله رو تین و کلاسیک هست و جواش همیشه که این undecidable است.
حالا مثلاً میخواهیم ما بیایم اکشن شون بدیم که $Halt_{TM}$ (ماشین تورینگی که در w توقف میابد یا نه)
آن دسایبل است. حون این سوال نخیته و همیشه ای نمیتونیم جواشو بدیم پس با استفاده از ریداکشن حل میکنیم
اینا undecidable بودن $HALT_{TM}$: $HALT_{TM}=\{\langle M,w\rangle \mid M$ متوقف میشه روی $w\}$
ایده کلی: فرض کنیم (برخلاف واقع) که یک ماشین تورینگ تصمیم گیرنده ای برای $HALT_{TM}$ وجود دارد
، یعنی تابعی که ما میگیریم ماشین M روی ورودی w متوقف میشه یا نه بهمون نشان میدهیم که اگر
چنین چیزی وجود داشته باشه آنگاه میتوان یک ماشین تصمیم گیرنده برای A_{TM} ساخت. ولی چون
ما میدانیم A_{TM} تصمیم پذیر نیست پس به تناقض میرسیم.

گام ① فرض کنیم یک ماشین تورینگ R وجود دارد که تصمیم میگیرد $HALT_{TM}$ را، یعنی:

$$R(\langle M,w\rangle)=\begin{cases}\text{yes} & \text{اگر } M \text{ روی } w \text{ متوقف شود}\\ \text{No} & \text{اگر متوقف نشود}\end{cases}$$

گام ② ساخت ماشین تصمیم گیرنده برای A_{TM} با استفاده از R : میخواهیم از این R استفاده
کنیم تا یک ماشین جدید D بسازیم که تصمیم میگیرد آیا M ، رشته w را میپذیرد یا نه. ماشین D کارش اینه:

① با R بررسی میکند که آیا M روی w متوقف میشه یا نه $A_{TM}\leq HALT_{TM}$

② اگر گفت نه ← پس M هرگز w را نمی پذیرد

③ اگر گفت بله ← حالا M را روی w اجرا میکنیم و خروجی $M(w)$ رو بررسی میکنیم: ① اگر $M(w)$ پذیرفت
D میگوید بله / اگر رد کرد D میگوید نه. در این حالت D یک تصمیم گیرنده برای A_{TM} است.
ولی مشکل اینه که ما میدانیم A_{TM} تصمیم ناپذیره X پس $HALT_{TM}$ هم تصمیم ناپذیره (خلاف فرض)
(نقیضه سگ هاروارد یا GPT و خروجه حجت نمون)

قضیه رایس: فرض کن یه تو یه ماشین تورینگ M میدن و ازت میپرسن: در آیا زبان این
ماشین خاصیت X رو داره؟ قضیه رایس میگه: اگر خاصیت x فقط برای بعضی از زبان ها
برقرار باشه و برای بعضی ها نه اونوقت تشخیص این خاصیت غیرممکن undecidable است.

Time complexity دو دسته مسائل داریم: ① Polynomial ② exponential

که اولی یعنی اوردر حل مساله آن حتما جمله ای است ولی دومی خیلی بد

ما میخواهیم با اولی مسائل رو حل کنیم.

مسائلی که با اولی حل میشن میگن مسائل P و مسائلی که با دومی حل میشن ولی نمیتونیم

کردن دومی از اوردر حتما جمله ای باشه رو میگیم مسائل NP.

Deterministic Polynomial ⟵ P ⊆ NP ⟶ Non-Deterministic Polynomial

رویکرد کاهش زمانی.

$w \in A \iff f(w) \in B$

A را به B کاهش میدهیم.

قضیه: اگر زبان A به زبان B به صورت جمله ای قابل کاهش باشد $(A \leq_p B)$

اگر B در زمان جمله ای قابل حل باشد پس A هم در زمان جمله ای قابل حل هست

اثبات: فرض کن M ماشین باشد که زبان B رو در زمان جمله ای حل میکند: میخواهیم ماشینی

طراحی کنیم (N) که زبان A رو هم در زمان جمله ای حل کند.

میدانیم $w \in A$ ، $f(w) \in B$ (f) رو باید پیدا کنیم، حال $f(w)$ رو میدهیم به M اگر

اکسپت کرد یعنی w در A است.

NP-complete: ① خودش در کلاس NP باشد. ② همه مسائل NP به اون در زمان جمله ای کاهش

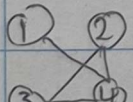
قضیه: اگر A NP-complete باشد، B NP باشد، واگر A را بتوان به B کاهش داد

آنگاه B NP-complete هست. واضح چون طبق تعریف B قابل کاهش به A هست

و A هم که به صورت سوال گفته به B کاهش پذیر هست پس B باید NP-complete باشد.

یک زبان NP-complete: (SAT): به عبارت بولین میده میگه متغیر هاروی مقدار بده

تا حاصلش TRUE شه.



clique: مجموعه راس هایی که زیرگراف کامل باشند: (1,4), (2,3,4), (1), ...

Indendent set: مستقل: (1,2), (1,3), (4)

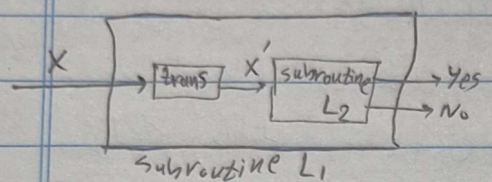
vertex cover: همسایه های گراف رو شامل میشوند: (2,4), (3,4), (1,2,3)

کلیک مسئله NP هست چون راحل این هم زیر مجموعه هارا چک کنیم کفایت.

ولی چک کردن این که آیا ظالمی شرط مساله رو داره یا نه توی Poly حل میشه

(معما چه حل شود آسان میشود)

for any $w \in \Sigma^*$, $w \in L_1$ if $f(w) \in L$.



if $L_1 \leq_p L_2$ and $L_2 \in P$, then $L_1 \in P$

if $L_1 \leq_p L_2$ and $L_2 \in NP$, than $L_1 \in NP$

if $L_1 \leq_p L_2$ and $L_1 \in P$, then $L_2 \notin P$

Theorem: $CLIQUE \leq_p INDEPENDENT\text{-}SET$ مثال

تبدیل: ① گراف G را بگیر ② گراف مکمل G را بساز ③ حالا ببین آیا گراف G یک مجموعه مستقل به اندازه k دارد؟

هر کلیک در گراف G معادل یک مجموعه مستقل در گراف مکمل G است.

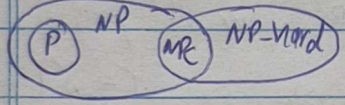
اگر G کلیک با اندازه k داشته باشد ← G هم مجموعه مستقل با اندازه k دارد.

اگر G کلیک با اندازه k نداشته باشد ← G هم مجموعه مستقل با اندازه k ندارد.

نتیجه: اگر بتوانیم INDEPENDENT-SET را حل کنیم، با این تبدیل میتوانیم کلیک را هم حل کنیم.

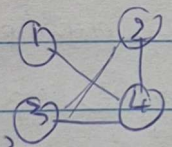
یک زبان L' NP-hard است اگر برای هر $L' \in NP$: $L \leq_p L'$

زبان L یک NP-complete است اگر L یک NP-hard باشد، $L \in NP$



SAT Problem: یک NP-complete است که یک به هر کدام از ترکیب های عبارت بولین چه مقداری بدهیم تا حاصل کل شود TRUE. حل این مسئله نمایی ولی آن verify آن Poly است

Inde: مثال: INDEPENDENT-SET $\leq_p$ VERTEX-COVER



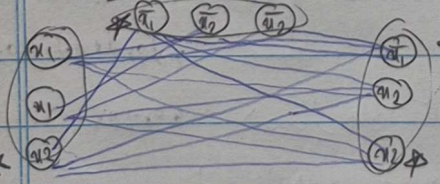
→ $\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1,2\}, \{1,3\}$

ماکمل کردن مجموعه رئوس در هر مجموعه ✓ مثال:

vertex ⇒ $\{2,4\}, \{3,4\}, \{1,2,3\}, \{1,2,4\}, \{1,3,4\}, \{2,3,4\}, \{1,2,3,4\}$

SAT → 3SAT → CLIQUE → IS → VC :مجموعه ریداکشن ها

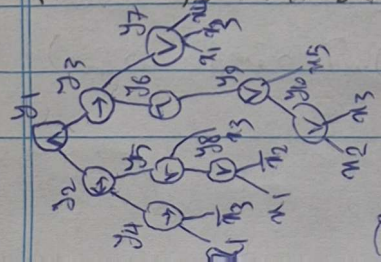
مثال: $3SAT \leq_p CLIQUE$ $\xrightarrow{\text{مثال}}$ $(x_1 \vee \bar{x}_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_2)$ (I) (II) (III) 3CNF



هر راس در هر دسته به همه راس های دسته های دیگر به شرطی که مکمل نباشد وصل میکنیم

اگر دوری دیدیم که از هر دسته فقط یک راس ✓

$((x_1 \rightarrow \bar{x}_3) \leftrightarrow (x_1 \vee \bar{x}_2 \vee x_3)) \wedge ((\overline{x_2 \vee x_3 \vee x_5}) \rightarrow (x_1 \vee x_3 \vee x_4)) \Leftarrow SAT \leq_p 3SAT$



$y_1 \leftrightarrow (y_2 \wedge y_3)$

$y_2 \leftrightarrow (y_4 \leftrightarrow y_5)$, ...

درخت سوخ میکشیم:

همه راس ها رو میدیم: y_1 ✓ میکشیم (مثل قبلی و بس)

همه را به همان y_1 وصل میکنیم

SOBHAN

همون درخت برای هر کدام میزنیم و ۵ تا رو میدیم و تمام میشه